Cape Peninsula University of Technology
creating futures

Department of Electrical Electronics and Computer Engineering

SDN260S – Software Design II

Group 14

Group Members

Name & Surname	Student number
Nontsikelelo Khumalo	222324007
Siyavuya Bonte	223025968
Syabonga Bonginkosi Nkuna	222015152
Ntokozo Eugine Sibiya	231208456
Ntsako Khosa	230654746
Regaugetswe Kgomara Mokgwatsana	230276210
Lesedi Oratile Mataboge	231199104
Simamkele Mayile	230131077

Table of Contents

Executive summary 2
1. Introduction and problem definition 2
2. Project objectives and scope 2
3. Development progress across check-ins 3
4. System architecture and UML design 3
4.1 Responsibilities of the main classes 5
5. Implementation and operation 5
5.1 Cryptographic hashing and baseline 5
5.2 Integrity comparison 5
5.3 Command-line interface 5
5.4 Graphical user interface 6
6. Testing methodology and results 7
7. Error handling, logging and usability 9
8. Challenges and engineering decisions 9
9. Graduate attribute reflections 9
GA2 – Application of scientific and engineering knowledge 9
GA7 – Sustainability and impact of engineering activity 10
10. Team contributions 10
11. Conclusion and recommendations 10

FINAL PROJECT REPORT — Java-Based Cryptographic File Integrity Monitor

Executive summary

The Java-Based Cryptographic File Integrity Monitor (FIM) is a desktop and console application developed to identify unexpected changes in monitored files. It creates a trusted baseline using SHA-256 hashes, saves the baseline for later use, and compares subsequent directory scans with the stored records. Results are categorized as unchanged, modified, new or deleted, with reports that users can review and export.

The project progressed from initial design and team allocation in Check-In 1, through a functioning console prototype in Check-In 2, to a GUI-supported and tested prototype in Check-In 3. The final check-in documents are nine passing functional and edge-case tests, including baseline persistence, normal verification, report export, invalid paths, added and deleted files, and missing GUI directory input. This report consolidates the documented evidence; it does not claim tests that were not separately demonstrated.

1. Introduction and problem definition

System administrators and everyday users need a practical way to recognize unauthorized file changes, accidental configuration edits and potentially malicious alteration of important data. Manual checking across directories is time-consuming and unreliable. The FIM addresses this problem by storing cryptographic file fingerprints and comparing them with a later scan. A mismatch indicates a change requiring investigation; it does not, on its own, identify the cause or prove malicious activity.

2. Project objectives and scope

- Select and validate a directory for monitoring.
- Recursively discover files and generate SHA-256 hashes.
- Create, persist and reload a baseline of file records.
- Compare a new scan with the baseline and categorize file changes.
- Present results through a CLI and a Java Swing GUI.
- Generate, review and export integrity reports; provide useful error messages and logs.

3. Development progress across check-ins

Milestone	Documented achievement
Check-In 1 – Planning	Problem statement, chosen solution, UML class diagram, group roles and three-week implementation plan.
Check-In 2 – Core prototype	CLI mode selection, directory validation, recursive scanning, SHA-256 hashing, baseline creation/loading, integrity checks, cached report viewing and report export. A 16-file baseline was demonstrated.
Check-In 3 – Integration and testing	Architecture explanation, Java Swing GUI, nine recorded passing tests using a three-file sandbox, addition/deletion detection, input validation, application logs and GA2/GA7 reflections.

4. System architecture and UML design

The application uses a modular object-oriented structure. Main initializes IntegrityService and provides access to ConsoleMenu and FileIntegrityGUI. Both interfaces delegate core operations to IntegrityService, which coordinates scanning, baseline persistence, comparison and reporting. This keeps user-interface code separate from the monitoring logic.

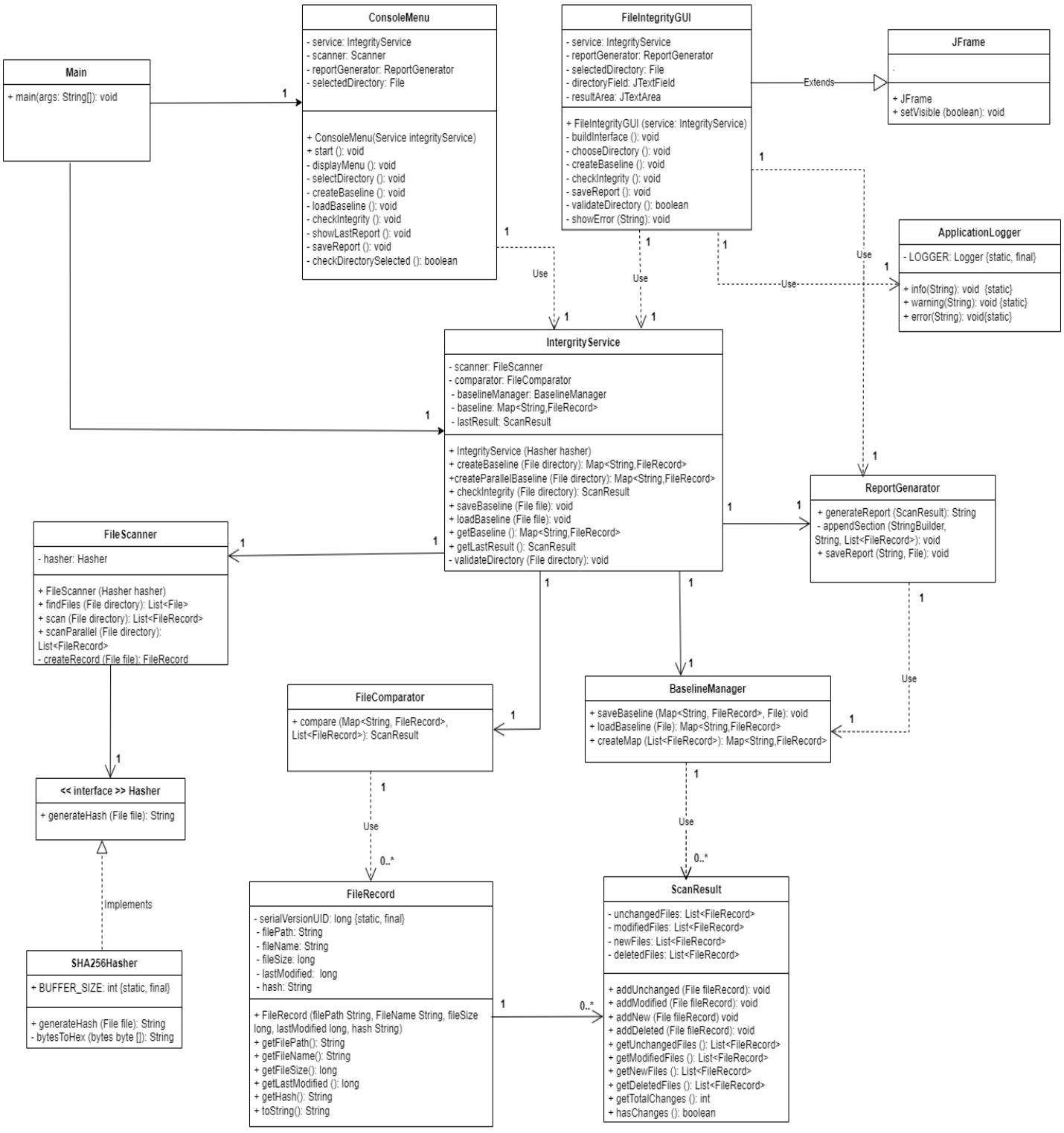


Figure 1. UML class diagram supplied in Check-In 3.

4.1 Responsibilities of the main classes

Class / interface	Responsibility
Main	Application entry point and interface selection.
ConsoleMenu / FileIntegrityGUI	CLI and Swing user interactions.
IntegrityService	Orchestrates baseline and integrity-check operations.
FileScanner	Recursively discovers files and collects records.
Hasher / SHA256Hasher	Defines hashing contract and computes SHA-256 digests.
FileRecord	Stores path, name, size, modification time and hash.
BaselineManager	Saves and loads collections of file records.
FileComparator / ScanResult	Compares records and groups unchanged, modified, new and deleted files.
ReportGenerator / ApplicationLogger	Formats/exports reports and records application events.

5. Implementation and operation

5.1 Cryptographic hashing and baseline

After a user selects a valid directory, FileScanner traverses its contents and uses SHA256Hasher to compute a digest for each file. FileRecord objects capture the associated metadata. BaselineManager persists the reference records so that the user can load them again in a later session. Check-In 2 documented creation and loading of 16 records; Check-In 3 repeated these operations with three files in a controlled test folder.

5.2 Integrity comparison

A new scan is compared with the baseline. A file present in both sets with the same hash is unchanged; a changed hash indicates modification; a newly present path is new; and a missing baseline path is deleted. The comparison output is stored in ScanResult and formatted into an integrity report. SHA-256 is used for change detection, not encryption or malware classification.

5.3 Command-line interface

The CLI menu provides seven options: Select directory, Create baseline, Load baseline, Check integrity, View last report, Save report and Exit. Check-In 2 showed that the most recent report can be viewed without repeating a scan and can be exported as a text file.

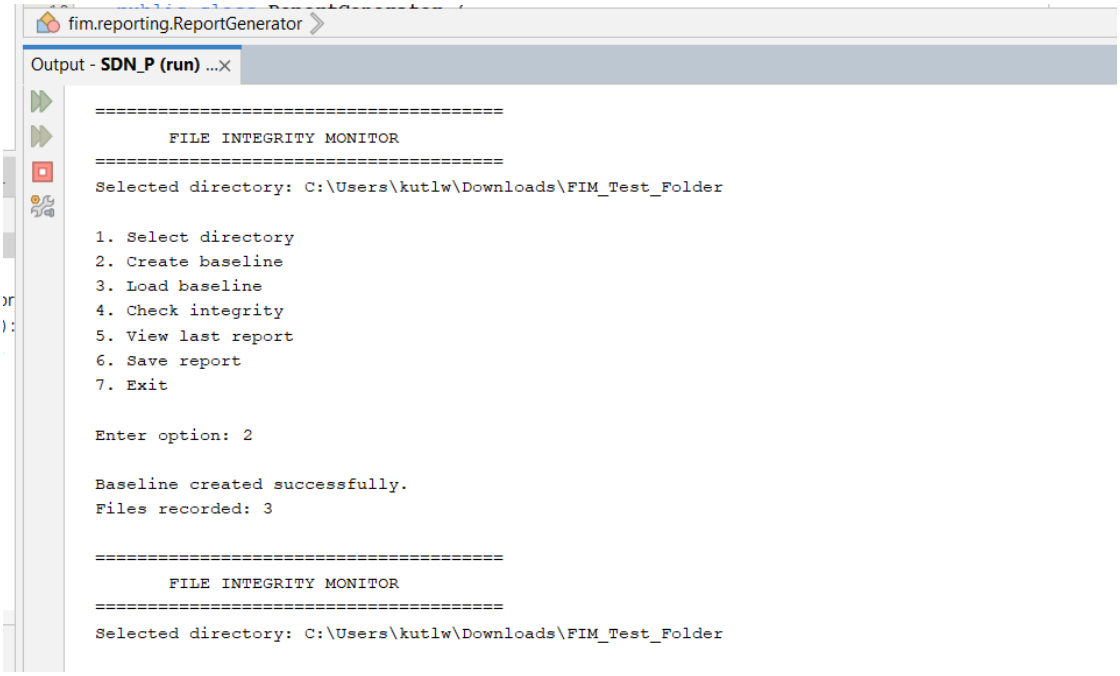


Figure 2. CLI baseline creation and baseline loading evidence from Check-In 3.

5.4 Graphical user interface

The Java Swing GUI provides a protected-folder field, Browse control, Create Baseline, Check Integrity, Save Report and Clear buttons, plus an output area. Folder selection and error dialogs reduce the need for users to type paths manually.

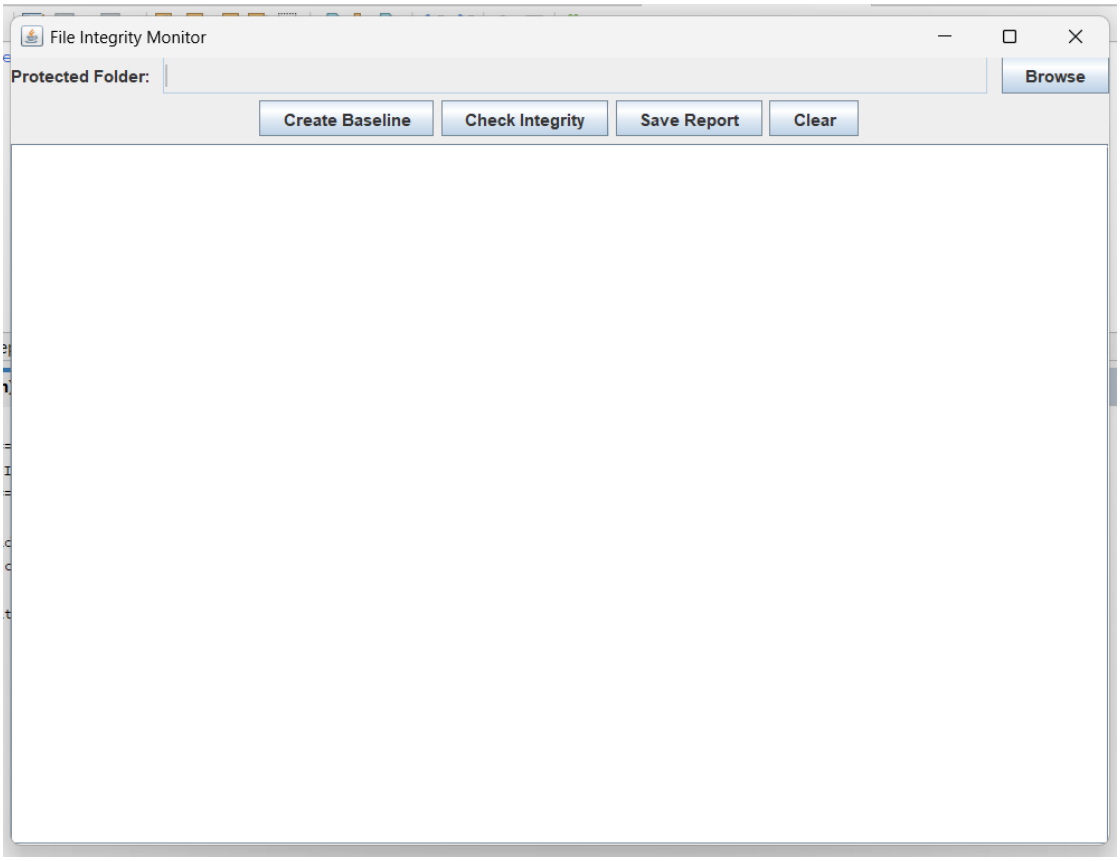


Figure 3. File Integrity Monitor graphical interface from Check-In 3.

6. Testing methodology and results

Check-In 3 records functional and edge-case testing using a controlled FIM_Test_Folder containing three baseline files. The test table below reproduces the documented outcomes in editable text rather than as an image.

ID	Test	Observed result	Status
1	CLI startup	Menu displayed; no directory selected	Pass
2	Baseline creation	Three files recorded	Pass
3	Data persistence	Three baseline records loaded	Pass
4	Normal verification	3 unchanged; 0 modified/new/deleted	Pass
5	Report export	Text report saved	Pass
6	Invalid directory	Error displayed; no crash	Pass
7	Added file	3 unchanged; 1 new	Pass
8	Deleted file	2 unchanged; 1 deleted	Pass
9	GUI empty input	“Please select a directory” dialog	Pass

All nine documented cases were marked Pass. The check-in also discusses modified-file detection as part of the implemented comparison logic, but the nine-row test table does not contain a separate modified-content test. The results therefore support the specific scenarios listed above rather than an exhaustive security assurance claim.

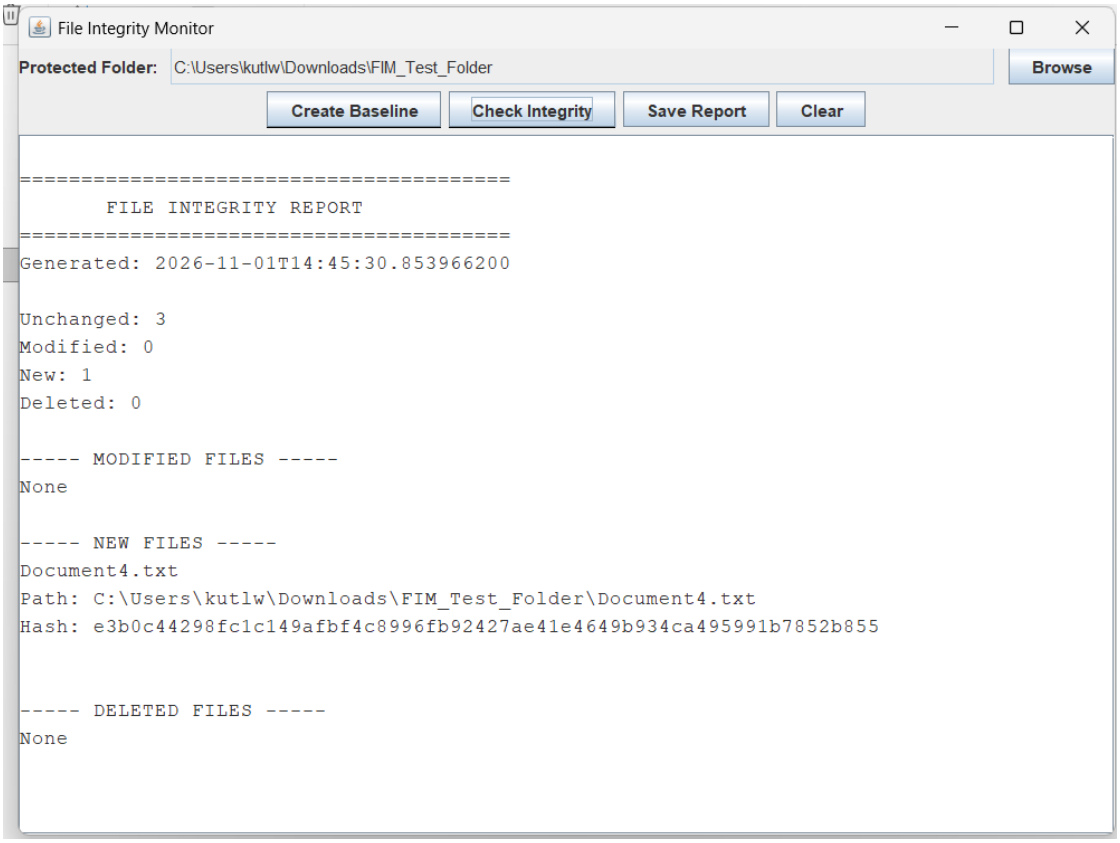


Figure 4. GUI output identifying a newly added file (Document4.txt).

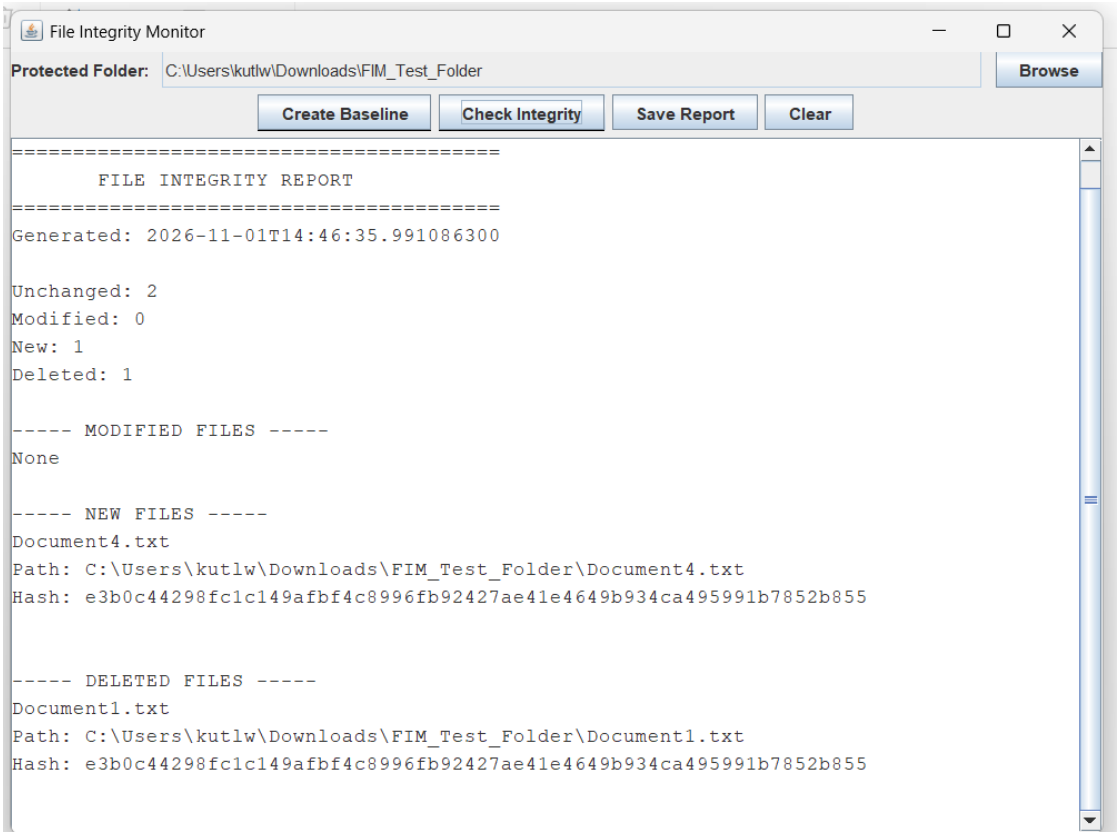


Figure 5. GUI integrity report showing new and deleted files.

7. Error handling, logging and usability

The prototype checks for invalid directories and missing folder selection before performing operations. CLI and GUI examples show user-friendly errors rather than uncontrolled termination. ApplicationLogger records timestamped events including baseline creation and integrity checks. These features support troubleshooting, audit review and more reliable interaction.

```
Sept 22, 2026 1:14:38 AM fim.logging.ApplicationLogger info
INFO: Creating parallel baseline.
Sept 22, 2026 1:14:38 AM fim.logging.ApplicationLogger info
INFO: Baseline saved to C:\Users\kutlw\OneDrive\Documentos\NetBeansProjects\SDN_P\data\baseline.dat
Sept 22, 2026 1:36:15 AM fim.logging.ApplicationLogger info
INFO: Creating parallel baseline.
Sept 22, 2026 1:36:16 AM fim.logging.ApplicationLogger info
INFO: Baseline saved to C:\Users\kutlw\OneDrive\Documentos\NetBeansProjects\SDN_P\data\baseline.dat
Sept 22, 2026 1:36:38 AM fim.logging.ApplicationLogger info
INFO: Baseline loaded. Files: 87
Sept 22, 2026 1:36:57 AM fim.logging.ApplicationLogger info
INFO: Integrity check started.
Sept 22, 2026 1:36:57 AM fim.logging.ApplicationLogger info
INFO: Integrity check completed.
Oct 18, 2026 1:29:19 AM fim.logging.ApplicationLogger info
INFO: Creating parallel baseline.
Oct 18, 2026 1:29:45 AM fim.logging.ApplicationLogger info
INFO: Baseline saved to C:\Users\kutlw\OneDrive\Documentos\NetBeansProjects\SDN_P\data\baseline.dat
Oct 18, 2026 1:39:31 AM fim.logging.ApplicationLogger info
INFO: Creating parallel baseline.
Oct 18, 2026 1:39:31 AM fim.logging.ApplicationLogger info
INFO: Baseline saved to C:\Users\kutlw\OneDrive\Documentos\NetBeansProjects\SDN_P\data\baseline.dat
Oct 18, 2026 1:39:55 AM fim.logging.ApplicationLogger info
INFO: Integrity check started.
Oct 18, 2026 1:39:55 AM fim.logging.ApplicationLogger info
INFO: Integrity check completed.
Oct 18, 2026 1:40:34 AM fim.logging.ApplicationLogger info
INFO: Creating parallel baseline.
Oct 18, 2026 1:40:35 AM fim.logging.ApplicationLogger info
INFO: Baseline saved to C:\Users\kutlw\OneDrive\Documentos\NetBeansProjects\SDN_P\data\baseline.dat
Oct 18, 2026 1:40:39 AM fim.logging.ApplicationLogger info
INFO: Creating parallel baseline.
Oct 18, 2026 1:40:39 AM fim.logging.ApplicationLogger info
INFO: Baseline saved to C:\Users\kutlw\OneDrive\Documentos\NetBeansProjects\SDN_P\data\baseline.dat
Oct 18, 2026 1:40:53 AM fim.logging.ApplicationLogger info
INFO: Baseline loaded. Files: 91
Oct 18, 2026 1:40:57 AM fim.logging.ApplicationLogger info
INFO: Integrity check started.
Oct 18, 2026 1:40:57 AM fim.logging.ApplicationLogger info
INFO: Integrity check completed.
```

Figure 6. Application log evidence from Check-In 3.

8. Challenges and engineering decisions

The check-ins identify practical concerns around directory validation, file-access exceptions, Windows file paths and integrating multiple components while preserving the UML design. The group used modular classes, interface-based hashing, validation before operations and a dedicated reporting layer to keep the prototype maintainable. Further testing on inaccessible directories, larger datasets and repeated modifications would strengthen the evidence.

9. Graduate attribute reflections

GA2 – Application of scientific and engineering knowledge

The project applies object-oriented design, Java collections, recursive traversal, file I/O, serialization, SHA-256 hashing, exception handling and Swing event-driven programming.

These concepts are integrated into a working monitoring workflow that links a theoretical model to measurable file-change results.

GA7 – Sustainability and impact of engineering activity

Automated comparison can reduce repetitive manual inspection. Modular components support maintenance and extension, while stored baselines and audit reports help responsible information management. The project also illustrates the importance of resource-conscious scanning and clear limitations: detecting a change does not automatically establish that it is harmful.

10. Team contributions

Member	Assigned responsibility (Check-In 1)
Ntsako Khosa	System design and UML architecture
Syabonga Nkuna	Core file records and hashing logic
Ntokozo Sibiya	Directory traversal and change detection
Lesedi Mataboge	Collections and baseline persistence
Siyavuya Bonte	Console interface and logging
Simamkele Mayile	Graphical user interface
Regaugetswe Mokgwatsana	Sandbox setup and tamper-verification testing
Nontsikelelo Khumalo	Project documentation, README and GA reflections

11. Conclusion and recommendations

The three check-ins demonstrate progression from design to a functional Java File Integrity Monitor with CLI and GUI interfaces, SHA-256 baseline creation and comparison, persistent storage, report export, logging and defensive input validation. The nine recorded tests passed in the controlled sandbox, including successful detection of added and deleted files.

Recommended next steps are a dedicated modified-file test, broader file-permission and large-directory testing, performance measurement, improved recovery from inaccessible files and an optional automated scanning schedule. These are recommendations, not features claimed as completed.